College of Computer
Science & Engineering

Department of Computer
Science & Artificial Intelligence

جامعة جدة
University of Jeddah

كلية علوم وهندسة الحاسب
قسم علوم الحاسب والذكاء الاصطناعي

Predicting Titanic Passenger Survival

CCAI312(Pattern Recognition)

Students Names
Futoon Abuzaid - 2211275
Selwan Halawani - 2210644
Albatool Al-Attas - 2210171

Dr. Safaa Al-Safri
First Semester 1446 A.H-2024 A.D.

ABSTRACT

This project aims to predict the survival of passengers aboard the Titanic using machine learning techniques. The Titanic dataset serves as a rich source of information, allowing for a comprehensive analysis of various factors influencing survival rates. The project begins with data preprocessing, which includes feature engineering, handling missing values, and encoding categorical variables.
We apply exploratory data analysis (EDA) to understand the dataset's characteristics and relationships between features. Three machine learning models XGBoost, K-Nearest Neighbors (KNN), and Decision Tree are trained and evaluated based on their accuracy, precision, recall, and ROC curves. Hyperparameter tuning is performed on the XGBoost model using GridSearchCV to optimize its performance.
The results reveal significant insights into the determinants of survival, showcasing the effectiveness of machine learning in predictive analytics. The project concludes with a discussion on model performance, overfitting analysis, and potential improvements for future work. This study not only emphasizes the power of machine learning in historical data analysis but also provides valuable lessons in data science methodologies.

KEYWORDS

Machine Learning, Classification, XGBoost, K-Nearest Neighbors (KNN), Decision Tree, Model Evaluation, Accuracy, Precision, Recall, F1-Score, ROC Curve, AUC (Area Under Curve), Hyperparameter Tuning, Performance Comparison, Metrics Analysis.

1.INTRODUCTION

The sinking of the Titanic is widely regarded as one of the most significant maritime disasters in history, leading to the tragic loss of numerous lives on April 15, 1912. The Titanic dataset offers a valuable repository of information, encompassing various passenger attributes, including age, gender, and ticket class. This dataset presents a unique opportunity to investigate the factors that influenced survival rates during this catastrophic event.
The primary objective of this project is to utilize machine learning techniques to analyze this data and develop predictive models for survival outcomes. By systematically examining the dataset and extracting critical features, we will construct and evaluate multiple models to determine their efficacy in predicting survival.

2.PROBLEM DEFINITION

The problem at hand is clearly defined as a classification task, specifically aimed at predicting the survival of passengers aboard the Titanic. The objective is to classify each passenger as either a survivor or a non-survivor based on various features.

3. RELATED WORK

Classification tasks are critical components in machine learning, with applications across domains like healthcare, finance, and natural language processing. Among the numerous models available, XGBoost has emerged as a standout algorithm due to its efficiency, scalability, and robust performance on complex datasets.
XGBoost (Extreme Gradient Boosting) is a gradient boosting framework that builds an ensemble of decision trees in a sequential manner, optimizing both speed and accuracy. Introduced by Chen and Guestrin in 2016, XGBoost gained popularity for its ability to handle large datasets, missing values, and complex feature interactions. Its advanced regularization techniques, such as L1 and L2 penalties, help prevent overfitting, making it a preferred choice for structured data problems.
Unlike classical models like Decision Tree or KNN, XGBoost leverages gradient descent to minimize errors across iterations, enabling it to capture nuanced patterns in data. Its support for parallel processing and distributed computing allows it to scale efficiently, even for high-dimensional datasets. Furthermore, XGBoost integrates seamlessly with hyperparameter tuning frameworks, such as GridSearchCV, to further optimize model performance.
In this project, XGBoost was chosen as a primary model for its proven capability to outperform traditional methods in both classification and regression tasks. Through hyperparameter tuning and evaluation on metrics like Accuracy, Precision, Recall, F1-Score, and AUC, this study aims to highlight XGBoost's ability to generalize effectively, even under challenging conditions. Comparing its performance to simpler models like KNN and Decision Tree further underscores its advantages in handling complex relationships within data.

4. DATA DESCRIPTION

The Titanic - Machine Learning from Disaster dataset from Kaggle contains information about passengers, aiming to predict survival based on various features.

Key Features

- Demographic: Sex (male/female), Age (with missing values).
- Socioeconomic: Pclass (1st, 2nd, 3rd), Fare.
- Family: SibSp (siblings/spouses), Parch (parents/children).
- Travel: Embarked (C, Q, S), Ticket, Cabin (mostly missing).

Target Variable

- Survived: Binary outcome (0 = Not Survived, 1 = Survived).

Data Characteristics

- Rows: 891
- Columns: 12 (reduced after preprocessing).
- Class Distribution: Imbalanced (62% Not Survived, 38% Survived).

Handling missing values and balancing the dataset were crucial for effective model training.

5. FEATURE ENGINEERING

Several challenges in the dataset were identified and addressed during the feature engineering process:

1. Missing Values:
   - Problem: Missing data in features like Age, Fare, and Embarked.
   - Solution:
     - Filled numerical features (Age, Fare) with their respective medians to reduce the impact of outliers.
     - Imputed categorical feature (Embarked) with the mode (most frequent value) to maintain consistency.

1. Categorical Variables:
   - Problem: Machine learning models cannot process categorical data directly.
   - Solution:
     - Applied binary encoding to Sex (male=0, female=1).
     - Used one-hot encoding for Embarked to represent categories without assuming any ordinal relationship.

- Irrelevant Features:
  - Problem: Features like PassengerId, Name, Ticket, and Cabin added noise without contributing to predictions.
  - Solution: Dropped these features to focus on relevant data.

- Feature Scaling:
  - Problem: Features like Age and Fare were on different scales, which could bias distance-based models (e.g., KNN).
  - Solution: Standardized numerical features using StandardScaler to normalize them for better model performance.
  -
- Imbalanced Data:
  - Problem: The target variable (Survived) was imbalanced, with more samples in Class 0 (Not Survived) than Class 1 (Survived).

These preprocessing steps addressed the dataset's challenges effectively, resulting in a clean, balanced, and optimized dataset ready for training robust machine learning models.

6. XGBOOST OVERVIEW

XGBoost is a powerful machine learning algorithm designed for structured data. It uses gradient boosting to build an ensemble of decision trees, effectively handling missing values, imbalanced data, and complex feature relationships.

How Data Was Prepared for XGBoost

1. Missing Values: Filled Age and Fare with medians and Embarked with the mode.
2. Categorical Encoding: Applied binary encoding for Sex and one-hot encoding for Embarked.
3. Imbalanced Data: Used oversampling to balance the Survived variable.
4. Feature Selection: Dropped irrelevant features like PassengerId, Name, and Cabin.

Suitable Data for XGBoost

- Works best with structured/tabular data containing numerical and categorical features.
- Excels in handling moderate-sized datasets with non-linear relationships.
- Handles imbalanced target variables effectively using built-in parameters like scale_pos_weight.

XGBoost's flexibility and efficiency made it a strong choice for this project.

8. STEPS TAKEN AFTER PREPROCESSING

.1 Splitting the Data

- Step: Divided the dataset into training and testing sets.
- Purpose: Ensures the models are trained on one subset and evaluated on unseen data to measure performance accurately.
- Ratio: Typically, 80% of the data was used for training, and 20% was reserved for testing.

2. Handling Imbalanced Data

- Step: Applied oversampling to balance the target variable (Survived) classes.
- Purpose: Ensures fair representation of both classes during training.
- as shown in Figure 1:
  - Before Balancing: Class 0 dominated the dataset.
  - After Balancing: Both classes were equal, enabling better model predictions.

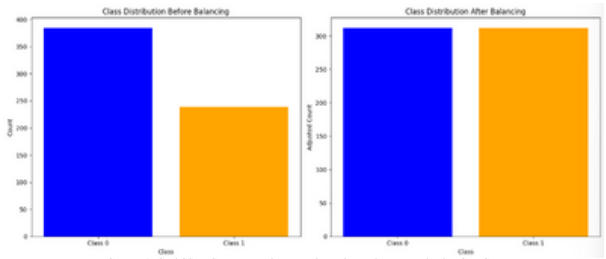


FIGURE 1: CLASS DISTRIBUTION BEFORE AND AFTER BALANCING

3. Training and Evaluating Models
Three models were trained and evaluated:

1. XGBoost:
   - A powerful gradient boosting algorithm for structured data.
   - Hyperparameters were tuned using GridSearchCV for optimal performance.
   - Evaluated metrics: Accuracy, Precision, Recall, F1-Score, and AUC.

2. K-Nearest Neighbors (KNN):
   - A distance-based algorithm that predicts based on neighboring samples.
   - Set k=5 as the number of neighbors for classification.

3. Decision Tree:
   - A simple tree-based model that splits data hierarchically.
   - Used default parameters for comparison.

As shown in the figure2, the graph demonstrates that XGBoost outperformed KNN and Decision Tree across all metrics, highlighting its superior performance and effectiveness.

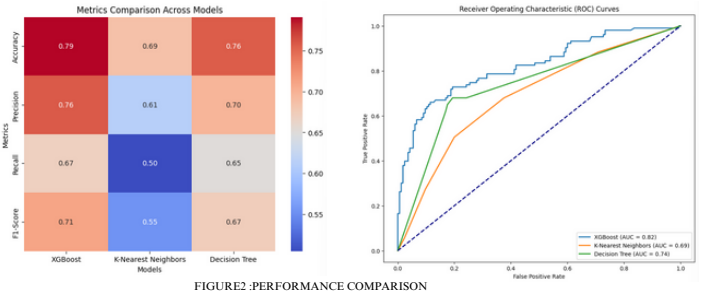


FIGURE2 :PERFORMANCE COMPARISON

Key Observations:

XGBoost Algorithm	KNN	Decision Tree
A powerful machine learning algorithm known for handling large datasets and achieving high accuracy.	A simple algorithm based on proximity, which may struggle with performance in larger datasets.	A widely used algorithm for classification that splits data based on specific attribute values.

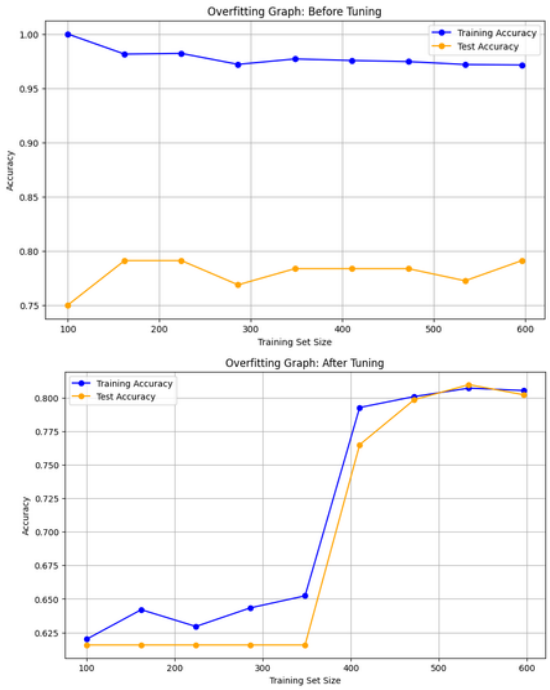


FIGURE 3: OVERFITTING GRAPHS BEFORE AND AFTER TUNING

From Figure 3: Overfitting Graphs Before and After Tuning, we can observe the following key comparisons:

Before Tuning:

Training Accuracy:
The training accuracy remains consistently high (close to 1.00), even with smaller training set sizes, which suggests the model has overfit the training data.

Test Accuracy:
The test accuracy is significantly lower compared to the training accuracy and does not improve consistently as the training set size increases. This indicates poor generalization.

After Tuning:
Training Accuracy:
The training accuracy shows a gradual increase as the training set size grows, demonstrating a more realistic learning curve and reducing signs of overfitting.
Test Accuracy:
The test accuracy aligns more closely with the training accuracy as the training set size increases, showing better generalization and a reduction in the overfitting gap.

Key Improvements After Tuning:

- **Reduced Overfitting:**

The significant gap between training and test accuracy in the "Before Tuning" graph is reduced, which indicates that the tuned model generalizes better to unseen data.

- **Better Generalization:**

The alignment of training and test accuracy in the "After Tuning" graph shows that the model is now learning patterns that generalize to the test set.

- **Improved Test Accuracy:**

The overall test accuracy has improved after tuning, which validates the effectiveness of the hyperparameter tuning process.

Visual Conclusion:
The tuning process successfully addressed overfitting as shown in figure 3 by introducing regularization and optimizing model parameters. This is evident from the smoother convergence of training and test accuracy curves in the "After Tuning" graph compared to the "Before Tuning" graph.

key tuning parameters

- ***n_estimators:*** The number of trees in the model that affect performance and the risk of overfitting.
- ***max_depth:*** Limits the depth of each tree, influencing model complexity.
- ***learning_rate:*** Controls the speed of learning, impacting model accuracy.
- ***subsample:*** Determines the fraction of data used for each iteration to mitigate overfitting.
- ***colsample_bytree:*** Specifies the fraction of features used to build each tree.
- ***gamma:*** Adds regularization to reduce overfitting by controlling tree splits.
- ***reg_lambda* and *reg_alpha:*** L2 and L1 regularization parameters, respectively, to enhance model generalization.

9. CHALLENGES

One of the primary challenges encountered during the implementation was related to the XGBoost model's handling of missing values. While XGBoost is designed to handle missing data by default, the model produced unexpected errors during training. This necessitated the manual preprocessing of the dataset, where missing values were imputed using appropriate strategies (e.g., median for numerical features). Although this additional step ensured smooth execution, it highlighted the need to carefully validate assumptions about model capabilities in practice.

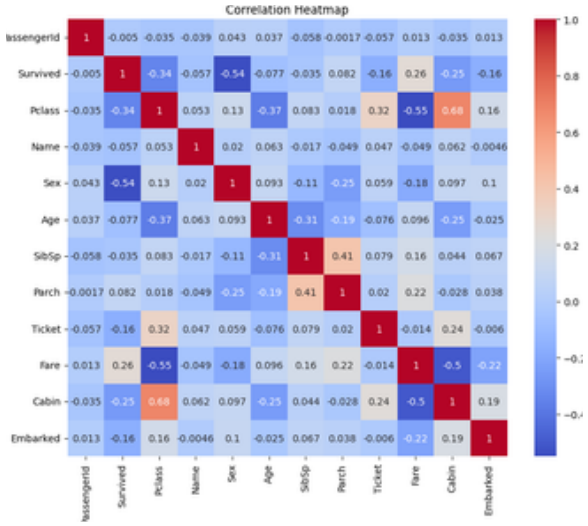
Another significant challenge was managing overfitting during hyperparameter tuning. Initial attempts with the default hyperparameter grid resulted in the model overfitting the training data, as evidenced by a significant performance gap between the training and validation sets. To address this, we modified the hyperparameter grid to include constraints such as reducing max_depth, increasing min_child_weight, and incorporating regularization parameters like lambda and alpha. These adjustments successfully mitigated overfitting, but required multiple iterations to identify the optimal configuration.

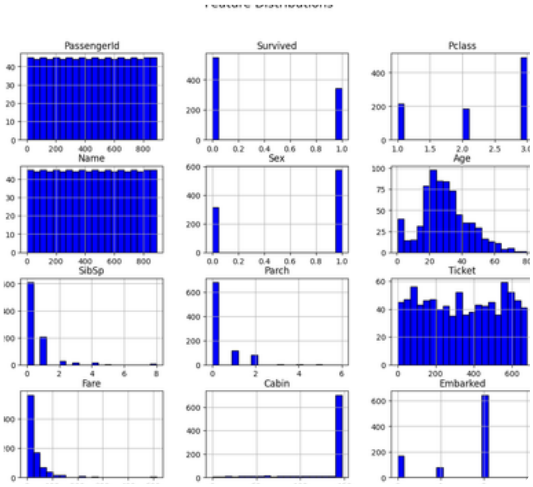
Additionally, determining the optimal data split posed another challenge. Multiple train-validation-test splits were tested to ensure balanced and representative training and evaluation. The process required experimenting with different split ratios and cross-validation strategies to achieve reliable and generalizable model performance.

10.BASIC STATISTICAL ANALYSIS

01 CORRELATION MATRIX

- Purpose: Shows relationships between numerical features.
- Insights: Identifies strongly correlated features and their relevance to the targe variable.



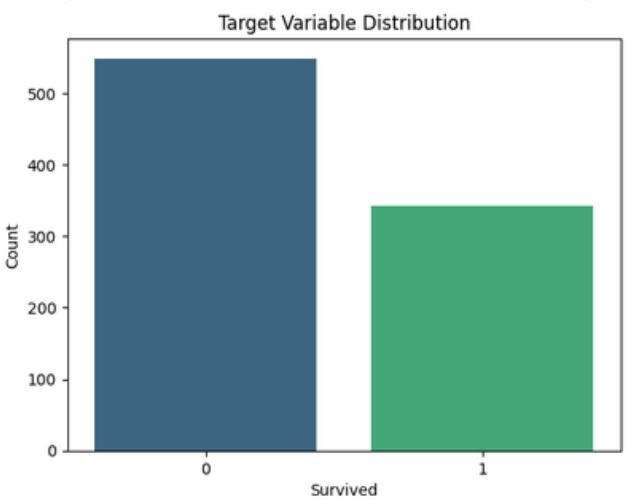


02 VISUALIZING NUMERIC FEATURE DISTRIBUTIONS

- Purpose: Displays the spread of numerical features like Age and Fare.
- Insights: Highlights outliers, skewness, and gaps in data that may need preprocessing.

03 TARGET VARIABLE DISTRIBUTION

- Purpose: Visualizes the class distribution of the target (Survived).
- Insights: Indicates class imbalance, requiring techniques like oversampling or weighting to improve model performance.



11. CONCLUSION

In this report, we explored the predictive modeling process for analyzing survival outcomes using the Titanic dataset. Key steps included feature engineering, handling data imbalances, hyperparameter tuning, and model evaluation.

1. **Feature Engineering:**

1.1 Features such as Title, Family Size, and IsAlone were engineered to extract meaningful patterns from the raw data.
1.2 Missing values in features like Age, Fare, and Embarked were imputed to ensure a complete dataset.

2. Model Training and Evaluation:
2.1 Multiple models, including XGBoost, K-Nearest Neighbors, and Decision Tree, were trained and evaluated.
2.2 The evaluation metrics included Accuracy, Precision, Recall, F1-Score, and AUC-ROC, ensuring a comprehensive assessment of model performance.

3. Hyperparameter Tuning:
3.1 Extensive hyperparameter tuning was performed for the XGBoost model to optimize its performance.
3.2 Regularization parameters such as gamma, reg_alpha, and reg_lambda were crucial in mitigating overfitting.

4. Performance Improvement:
4.1 Post-tuning, the XGBoost model demonstrated significant improvement in generalization with reduced overfitting. The gap between training and test performance decreased, and test metrics like AUC-ROC and F1-Score showed notable improvement.

5. Handling Data Imbalance:
5.1 Techniques such as SMOTE and class weighting effectively addressed class imbalances, improving model robustness in predicting minority class outcomes.

12. FINAL OBSERVATIONS

The XGBoost model emerged as the best-performing algorithm, balancing high accuracy and generalization.

While precision and recall were well-balanced, there remains room for improvement by experimenting with more advanced features or alternative algorithms such as Ensemble Models.

This project highlights the importance of a structured approach in predictive modeling, from data preprocessing to advanced evaluation techniques. Future work can explore additional techniques like deep learning or feature selection to further enhance model performance.

13. REFRENCES

[1] Kaggle, Titanic - Machine Learning from Disaster, 2024. Accessed: Nov. 29, 2024. [Online]. Available: https://www.kaggle.com/c/titanic

[2] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.

[3] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), New York, NY, USA, 2016, pp. 785–794. DOI: https://doi.org/10.1145/2939672.2939785.

[5] Scikit-learn Documentation, 2024. Accessed: Nov. 29, 2024. [Online]. Available: https://scikit-learn.org.